

THE CHORUS FABRIC: HIGH-DIMENSIONAL SIGNAL ORCHESTRATION FOR MACHINE-TO-MACHINE COMMUNICATION

FIELD OF THE INVENTION

The present invention relates to machine-to-machine (M2M) communication protocols for artificial intelligence systems, and more particularly to a method and system for transmitting high-dimensional tensor vectors directly between AI nodes using a cryptographically secured, multiplexed gRPC streaming fabric that eliminates text tokenization overhead.

BACKGROUND OF THE INVENTION

Current AI-to-AI communication relies predominantly on two paradigms, both of which introduce fundamental inefficiencies:

(1) HTTP/REST with JSON serialization. In this approach, an AI agent serializes its internal state or instructions into text, transmits that text via HTTP, and a receiving AI agent parses the text back into a usable representation. A 128-dimensional float32 embedding vector (512 bytes of actual data) requires approximately 1,420 bytes when encoded as a JSON array, representing a 2.8-fold expansion of the raw payload. Additionally, every HTTP request resends 400–600 bytes of protocol headers regardless of payload size. When no persistent connection is maintained, TLS handshakes add approximately 4,096 bytes per session. The cumulative overhead per message is 4.3x to 11.9x greater than the raw data payload.

(2) Large Language Model (LLM) API calls. In this paradigm, an AI agent converts its internal embedding-space state into natural language tokens, transmits those tokens via HTTP to a remote

LLM inference endpoint, and awaits a text response that a downstream AI must re-parse. This introduces full tokenization and detokenization cycles, LLM inference latency of 1–3 seconds, and the lossy semantic compression inherent in converting continuous embedding vectors to discrete text tokens. A single LLM API call to convey a 128-dimensional signal requires approximately 3,900 bytes and 1,000–3,000 milliseconds of total round-trip time.

(3) Absence of native AI signal authentication. Neither existing paradigm provides cryptographic authentication tied to the content of individual AI signals. Bearer tokens and API keys authenticate the connection, not the payload. A compromised relay node can inject or modify signals without detection by the receiving AI node.

(4) Absence of signal multiplexing. No existing protocol provides a mechanism for two AI agents to simultaneously transmit distinct signals over a shared channel such that each agent can recover only its own signal, or to holographically blend signals so that a single transmission carries the collective state of multiple agents.

There exists a need for a communication fabric specifically designed for AI-to-AI signal transmission that: (a) operates natively in embedding space without tokenization; (b) provides cryptographic authentication at the individual message level tied to both session identity and sequence number; (c) supports both isolated and superposed multi-agent transmission over a shared channel; and (d) enables relay nodes to amplify and audit signals without decrypting them, preserving end-to-end confidentiality.

SUMMARY OF THE INVENTION

The present invention provides a communication system and method, referred to herein as the CHORUS Fabric, comprising the following novel elements:

A **tensor transmission protocol** wherein AI signals are represented as high-dimensional float32 vectors and transmitted as packed binary payloads over bidirectional gRPC streams, bypassing text serialization entirely.

A **tensor multiplication cipher** wherein each payload vector V_{raw} is encrypted by matrix multiplication with a session key matrix K to produce ciphertext $V_{\text{enc}} = V_{\text{raw}} @ K$, and decrypted by the receiving node using the inverse key matrix K_{inv} such that $V_{\text{dec}} = V_{\text{enc}} @ K_{\text{inv}}$, where K and K_{inv} are generated via QR decomposition to guarantee numerical invertibility.

A **rolling neural watermark authentication scheme** wherein each payload is authenticated by injecting a deterministic unit vector derived from $\text{SHA-256}(\text{session_id} \parallel \text{sequence_number})$ prior to encryption, and verified at the receiving node by computing cosine similarity between the decrypted vector and the expected watermark vector, rejecting payloads below a configurable similarity threshold.

An **orthogonal isolation mode (Mode A)** wherein two transmitting nodes share a single encrypted channel without signal crosstalk, by projecting their respective signals onto orthogonal subspaces W_A and W_B where the product $W_A @ W_B$ approximates the zero matrix, mixing the projections into a single tunnel vector, and recovering each signal independently at the receiver by re-applying the respective projection matrix.

A **holographic superposition mode (Mode B)** wherein two or more transmitting nodes contribute signals that are arithmetically summed into a single collective vector $V_{\text{collective}} = V_A + V_B$, allowing the receiving node to observe the consensus or aggregate state of the contributing agents.

A **relay node architecture** wherein an intermediate node amplifies a ciphertext payload by scalar multiplication $V_{\text{amp}} = \text{scalar} \times V_{\text{enc}}$ without holding the decryption key, logging a

cryptographic fingerprint $\text{SHA-256}(V_{\text{enc}})$ for audit purposes, and preserving end-to-end confidentiality of the plaintext signal.

A **control plane key issuance service** that generates and distributes ephemeral session key bundles (K , K_{inv} , watermark_seed) to registered nodes upon request, with configurable time-to-live, decoupling key management from signal transmission.

BRIEF DESCRIPTION OF THE DRAWINGS

FIG. 1 is a system architecture diagram illustrating the four-node CHORUS Fabric topology comprising a Control Plane, a Source Pod (Transmitting Node), a Relay Node, and a Target Pod (Receiving Node), showing gRPC stream connections and IP addresses from a live transatlantic deployment.

FIG. 2 is a flow diagram illustrating the signal watermark injection and encryption process at the transmitting node, including SHA-256 watermark generation, alpha-scaled injection, L2 normalization, and matrix multiplication encryption.

FIG. 3 is a flow diagram illustrating the decryption, watermark verification, and mode dispatch process at the receiving node, including the cosine similarity decision gate.

FIG. 4 is a diagram illustrating Mode A Orthogonal Isolation, showing projection of two agent signals onto orthogonal subspaces, tunnel vector formation, transmission, and independent signal recovery at the receiver.

FIG. 5 is a diagram illustrating Mode B Holographic Superposition, showing arithmetic combination of two agent signals and approximate recovery of each contributing signal at the receiver.

FIG. 6 is a diagram illustrating the Relay Node architecture, showing scalar amplification of ciphertext, SHA-256 fingerprint audit logging, and forwarding to the target node without access to the decryption key.

FIG. 7 is a sequence diagram of the Control Plane key issuance lifecycle, including node registration, session key bundle generation, TTL-governed expiry, and re-registration flow.

FIG. 8 is a benchmark results chart showing measured per-chunk round-trip latency distributions and bandwidth comparisons for all three transmission modes across a live transatlantic deployment (US East to Germany West Central, 7,766 total transmissions).

DETAILED DESCRIPTION OF THE PREFERRED EMBODIMENTS

The following detailed description sets forth specific embodiments of the CHORUS Fabric. The embodiments described herein are illustrative and not intended to limit the scope of the invention as defined by the claims.

I. System Architecture

Referring to *FIG. 1*, the CHORUS Fabric comprises four primary node types deployed as networked services communicating over gRPC using Protocol Buffer-encoded messages.

(a) Control Plane Node

The Control Plane Node 100 is a key issuance service that maintains a registry of active sessions. Upon receiving a registration request from a transmitting or receiving node, the Control Plane generates an ephemeral key bundle comprising: an encryption matrix K of dimension $d \times d$; a decryption matrix K_{inv} of dimension $d \times d$; a watermark seed string; a universally unique session identifier; and a session expiry timestamp equal to the current time plus a configurable time-to-live (TTL). In a preferred embodiment the TTL is 7,200 seconds. The Control Plane exposes three gRPC remote procedure calls: RegisterAndRequestKey, RequestOrthogonalProjections, and GetRelayInstruction.

(b) Source Pod (Transmitting Node)

The Source Pod 110 is an AI agent node that generates high-dimensional signal vectors and transmits them to a downstream target. The Source Pod registers with the Control Plane to obtain a session key bundle, constructs payload vectors according to the selected transmission mode (Direct, Isolation, or Superposition), injects a rolling neural watermark, encrypts the vector using the session key, and initiates a bidirectional gRPC stream to either the Relay Node or the Target

Pod directly. In the preferred embodiment, Source Pods are deployed as containerized Python services.

(c) Relay Node

The Relay Node 120 is an intermediate forwarding node that receives ciphertext payload streams, applies linear scalar amplification to each ciphertext vector without decrypting it, logs a SHA-256 cryptographic fingerprint of each payload for audit purposes, and forwards the amplified ciphertext to the Target Pod. Critically, the Relay Node does not hold the decryption key K_{inv} at any time, preserving end-to-end confidentiality between the Source Pod and the Target Pod. The Relay Node exposes a bidirectional gRPC streaming RPC (RelayStream) and employs a background thread with a bounded queue for collecting downstream acknowledgements.

(d) Target Pod (Receiving Node)

The Target Pod 130 is an AI agent node that receives encrypted payload streams over gRPC, decrypts each payload using session key K_{inv} obtained from the Control Plane, verifies the rolling neural watermark, dispatches the decrypted signal to a mode-specific handler, and returns per-payload acknowledgements containing the sequence number, verification status, signal norm, and processing status. The Target Pod exposes a bidirectional gRPC streaming RPC (StreamSignal).

II. Tensor Multiplication Cipher

The CHORUS Fabric employs a tensor multiplication cipher for payload encryption. This cipher is specifically suited to high-dimensional float32 vectors because the encryption operation (matrix multiplication) is structurally identical to the linear transformations performed inside neural network inference pipelines, enabling hardware acceleration on existing AI processors without dedicated cryptographic co-processors.

Key Generation

Given a signal dimension d , the Control Plane generates a random matrix A of shape (d, d) with entries drawn from a standard normal distribution. QR decomposition is applied to A to obtain an orthogonal matrix Q . A random positive diagonal scaling matrix S is generated with diagonal entries drawn uniformly from a range ensuring a well-conditioned key. The encryption key is defined as:

$$K = Q @ S$$

The decryption key is:

$$K_{\text{inv}} = S_{\text{inv}} @ Q_{\text{transpose}}$$

where S_{inv} is the elementwise reciprocal of the diagonal of S . The product $K @ K_{\text{inv}}$ equals the identity matrix I within numerical floating-point precision. In an experimental implementation with $d=128$, the identity reconstruction error $\|K @ K_{\text{inv}} - I\|$ was measured at 1.86×10^{-15} . All key arithmetic is performed in float64 precision; keys are downcast to float32 only for wire transmission.

Encryption

Given plaintext signal vector V_{raw} of shape $(d,)$ and encryption key matrix K of shape (d, d) :

$$V_{\text{enc}} = \text{cast_to_float32}(\text{cast_to_float64}(V_{\text{raw}}) @ \text{cast_to_float64}(K))$$

Decryption

Given ciphertext vector V_{enc} of shape $(d,)$ and decryption key matrix K_{inv} of shape (d, d) :

$$V_{\text{dec}} = \text{cast_to_float32}(\text{cast_to_float64}(V_{\text{enc}}) @ K_{\text{inv}})$$


```
cast_to_float64(K_inv) )
```

III. Rolling Neural Watermark

The rolling neural watermark provides per-payload cryptographic authentication tied to both the session identity and the message sequence number, as illustrated in *FIG. 2* and *FIG. 3*.

Watermark Vector Generation

For a session with watermark seed string ws and a payload with sequence number n , the expected watermark unit vector $W(n)$ is generated as follows:

```
hash_bytes = SHA-256( ws.encode('utf-8') + str(n).encode('utf-8') )

seed_int = int.from_bytes(hash_bytes, byteorder='big') mod
(2^64)

R = random_vector(dim=d, seed=seed_int) // seeded PRNG

W(n) = R / ||R|| // L2 normalization
```

Watermark Injection

Prior to encryption, the watermark is injected into the signal vector:

```
V_wm = normalize( V_raw + alpha × W(n) )
```

where α is an injection strength hyperparameter (default 0.1 in the preferred embodiment) and normalize denotes L2 normalization to the unit sphere. The watermarked vector V_{wm} is then encrypted as described in Section II.

Watermark Verification

At the receiving node, following decryption of V_{dec} , the watermark is verified by computing the cosine similarity between the decrypted vector and the expected watermark vector $W(n)$ recomputed from the shared seed and the received sequence number:

```
similarity = dot(V_dec, W(n)) / ( ||V_dec|| × ||W(n)|| )

verified = ( similarity >= threshold ) // default threshold =
0.95
```

Payloads with similarity below the threshold are rejected. This mechanism prevents replay attacks because each sequence number n produces a distinct $W(n)$, making replayed payloads from earlier sequence numbers fail verification. It prevents injection attacks because generating a valid watermark requires knowledge of the session-specific seed ws . In the experimental transatlantic deployment, 7,766 consecutive payloads across 13 independent benchmark runs achieved 100% watermark verification.

IV. Mode A: Orthogonal Isolation

Mode A enables two AI agents (Agent A and Agent B) to share a single encrypted channel while transmitting independent signals, such that each agent recovers only its own signal at the receiving node, as illustrated in *FIG. 4*.

Projection Pair Generation

The Control Plane generates a pair of projection matrices (W_A , W_B) of shape (d, d) via QR decomposition of a random matrix, arranged such that:

```
W_A @ W_B_transpose ≈ 0 (zero matrix)
```

In the preferred embodiment with $d=128$, the maximum absolute element of the product $W_A @ W_B_{\text{transpose}}$ has been measured at less than 1×10^{-6} . These projection matrices are transmitted

to both transmitting and receiving nodes as part of the Mode A session bundle.

Signal Projection and Mixing at Transmitter

```
P_A = W_A @ V_A // project Agent A signal onto subspace W_A
```

```
P_B = W_B @ V_B // project Agent B signal onto subspace W_B
```

```
V_tunnel = normalize(P_A + P_B) // form tunnel vector
```

V_tunnel is then watermarked and encrypted per Sections III and II before transmission.

Signal Recovery at Receiver

After decryption and watermark verification of V_dec :

```
V_A_recovered = W_A @ V_dec
```

```
V_B_recovered = W_B @ V_dec
```

The cross-signal crosstalk, defined as $\|W_A @ V_B\| / \|V_B\|$, was experimentally measured at 0.000006% for $d=128$. The recovery cosine similarity between $V_A_recovered$ and the original V_A was measured at 1.0 (perfect reconstruction to floating-point precision).

V. Mode B: Holographic Superposition

Mode B enables two or more AI agents to holographically blend their signals into a single transmission vector representing their aggregate state, as illustrated in *FIG. 5*.

Signal Superposition

```
V_collective = normalize(V_A + V_B)
```

$V_collective$ is then watermarked and encrypted per Sections III and II before transmission.

Signal Recovery

After decryption, the approximate contribution of each agent can be estimated by projecting V_{dec} onto the original signal directions. In experimental measurements with two equal-magnitude contributors and $d=128$, the cosine similarity between V_{dec} and each of V_A and V_B was approximately 0.70. Mode B is suited for applications where the collective or consensus AI state is the primary signal of interest, such as multi-agent voting, ensemble model agreement, or distributed sensor fusion.

VI. Relay Node Architecture

The Relay Node provides signal amplification and audit logging without compromising end-to-end confidentiality, as illustrated in *FIG. 6*. This architecture is analogous to a photonic amplifier in optical fiber networks.

Ciphertext Amplification

Upon receiving ciphertext vector V_{enc} , the Relay Node applies:

$$V_{amp} = \text{amplification_factor} \times V_{enc}$$

This operation is mathematically safe on ciphertext because decryption is a linear operation:

$$\begin{aligned} \text{decrypt}(V_{amp}) &= (\text{amplification_factor} \times V_{enc}) @ K_{inv} = \\ &\text{amplification_factor} \times (V_{enc} @ K_{inv}) = \text{amplification_factor} \\ &\times V_{raw_watermarked} \end{aligned}$$

The receiving node compensates for the amplification factor using a pre-shared relay instruction obtained from the Control Plane via the GetRelayInstruction RPC.

Audit Fingerprinting

For each relayed payload, the Relay Node computes and stores:

```
fingerprint = SHA-256( V_enc.tobytes() )
```

This fingerprint provides a tamper-evident audit log of all relayed ciphertexts without revealing any information about the plaintext content.

VII. Control Plane Key Management

The Control Plane implements Option B key management, illustrated in *FIG. 7*, wherein key generation is centralized in a dedicated ephemeral service separate from the signal transmission path.

Session Lifecycle

- (1) A Source Pod sends a PodRegistration message containing its pod_id, requested session parameters, and mode flags.
- (2) The Control Plane generates a SessionKeyBundle containing: session_id (UUID v4), K (serialized as bytes), K_inv (serialized as bytes), watermark_seed (32-character hex string), expires_at (Unix timestamp = current_time + TTL), and optionally W_A, W_B matrices for Mode A sessions.
- (3) The SessionKeyBundle is returned to the Source Pod over the RegisterAndRequestKey RPC.
- (4) The corresponding Target Pod obtains its bundle (including K_inv) using the session_id as a rendezvous token.
- (5) Upon TTL expiry, both nodes must re-register; payloads using expired session keys fail watermark verification.

Key Serialization

Key matrices are serialized as raw float32 bytes ($d \times d \times 4$ bytes per matrix) for transmission. Upon receipt, matrices are deserialized and upcast to float64 for all arithmetic operations.

VIII. Wire Protocol

Each CHORUS payload is transmitted as a TensorPayload Protocol Buffer message comprising: data (bytes — packed float32 array of length $d \times 4$), dim (int32 — embedding dimension), seq_len (int32), pod_id (string), session_id (string), mode (string — "direct", "isolation", or "superposition"), watermark (bytes — packed float32 watermark vector), and seq_num (int32 — monotonically increasing sequence counter).

Three gRPC services are defined: ControlPlane (key issuance), TargetPod exposing StreamSignal as a bidirectional streaming RPC of type (stream TensorPayload) $\rightarrow$ (stream SignalAck), and RelayNode exposing RelayStream as a bidirectional streaming RPC of type (stream TensorPayload) $\rightarrow$ (stream RelayAck).

IX. Experimental Results

The CHORUS Fabric was deployed as a four-container system on Microsoft Azure Container Instances across two geographic regions: US East (Virginia) and Germany West Central (Frankfurt), representing a transatlantic deployment path of approximately 8,000 km.

Container endpoints: Control Plane at chorus-cp-de.germanywestcentral.azurecontainer.io:50051 (public IP 4.182.111.109); Target Pod at chorus-target-de.germanywestcentral.azurecontainer.io:50053 (public IP 98.67.201.195); Relay Node at chorus-relay-us.eastus.azurecontainer.io:50052 (public IP 20.237.21.245).

A benchmark of 7,766 total payload transmissions was conducted across 13 independent runs, each comprising 200 payloads per mode with 5 warmup payloads discarded. Results are shown in *FIG. 8*:

- Direct mode: p50 RTT = 179 ms, p95 = 300 ms, p99 = 1,596 ms
- Isolation (Mode A): p50 RTT = 311 ms, p95 = 381 ms, p99 = 1,685 ms
- Superposition (Mode B): p50 RTT = 1,274 ms, p95 = 1,422 ms

- Watermark verification: 7,766/7,766 (100%) across all modes and runs
- Mode A crosstalk: 0.000006%; Mode A recovery cosine similarity: 1.0
- Key invertibility identity error: 1.86×10^{-15}

The Direct mode p50 RTT of 179 ms equals the theoretical minimum for the physical transatlantic round-trip distance, confirming that the tensor multiplication cipher and watermark injection add no measurable latency overhead. Bandwidth consumption per payload was 548 bytes, compared to 2,440 bytes for equivalent HTTP/REST JSON communication (4.45× reduction) and 3,900 bytes for equivalent LLM API communication (7.1× reduction).

CLAIMS

(Note: Claims in a provisional application are optional and non-binding. These claims are provided to define the scope of the invention and should be reviewed and refined with qualified patent counsel before filing the non-provisional application.)

1. A computer-implemented method for machine-to-machine communication between artificial intelligence nodes, the method comprising:

generating, at a first AI node, a signal vector of dimension d representing internal state or instructions of the first AI node, wherein the signal vector comprises d floating-point numeric values;

obtaining, from a key issuance service, an encryption key matrix K of shape $d \times d$;

encrypting the signal vector by computing a matrix product $V_{enc} = V_{raw} @ K$;

transmitting V_{enc} as a binary payload over a bidirectional streaming remote procedure call to a second AI node; and

at the second AI node, obtaining a decryption key matrix K_{inv} and recovering the original signal by computing $V_{dec} = V_{enc} @ K_{inv}$;

wherein the signal vector is transmitted without conversion to text tokens at any point in the transmission path.

2. The method of claim 1, further comprising:

prior to encryption, computing a watermark unit vector $W(n)$ as a function of a session-specific seed and a monotonically increasing sequence

number n , wherein $W(n)$ is derived from a cryptographic hash of the concatenation of the seed and n ;

injecting the watermark by computing $V_{wm} = \text{normalize}(V_{raw} + \alpha \times W(n))$;

at the receiving node, recomputing $W(n)$ from the shared seed and received sequence number; and

verifying the payload by computing cosine similarity between V_{dec} and $W(n)$ and rejecting the payload if the similarity is below a threshold.

3. The method of claim 1, wherein the encryption key matrix K and decryption key matrix K_{inv} are generated by:

performing QR decomposition on a random matrix to obtain an orthogonal matrix Q ;

generating a positive diagonal scaling matrix S ; and

computing $K = Q @ S$ and $K_{inv} = S_{inv} @ Q_{transpose}$;

wherein $K @ K_{inv}$ equals the identity matrix within floating-point numerical precision.

4. The method of claim 1, further comprising:

generating a pair of projection matrices W_A and W_B such that $W_A @ W_B_{transpose}$ approximates the zero matrix;

at a first transmitting agent, projecting signal V_A to obtain $P_A = W_A @ V_A$;

at a second transmitting agent, projecting signal V_B to obtain $P_B = W_B @ V_B$;

forming a tunnel vector $V_{\text{tunnel}} = \text{normalize}(P_A + P_B)$ and transmitting V_{tunnel} as a single payload; and

at the receiving node, independently recovering $V_A_{\text{recovered}} = W_A @ V_{\text{dec}}$ and $V_B_{\text{recovered}} = W_B @ V_{\text{dec}}$;

wherein the two signals are recoverable independently without knowledge of each other.

5. The method of claim 1, further comprising:

forming a collective vector $V_{\text{collective}} = \text{normalize}(V_A + V_B)$ from signal vectors contributed by two or more AI agents; and

transmitting $V_{\text{collective}}$ as a single payload representing the aggregate state of all contributing agents.

6. A system for cryptographically authenticated machine-to-machine AI signal relay, the system comprising:

a relay processor configured to receive an encrypted signal vector V_{enc} transmitted from a source AI node;

apply a scalar amplification factor to produce $V_{\text{amp}} = \text{factor} \times V_{\text{enc}}$ without performing decryption of V_{enc} ;

compute a cryptographic fingerprint of the received ciphertext as $\text{SHA-256}(V_{\text{enc}}.\text{tobytes}())$ and store the fingerprint in an audit log; and

forward V_{amp} to a target AI node;

wherein the relay processor does not possess or receive a decryption key for V_{enc} at any time, such that the plaintext content of V_{enc} remains confidential from the relay processor.

7. The system of claim 6, wherein scalar multiplication of ciphertext is equivalent, after decryption at the target AI node by the target's decryption key K_{inv} , to scalar multiplication of the original plaintext signal, such that $decrypt(V_{amp}) = factor \times decrypt(V_{enc})$, enabling the receiving node to compensate for the amplification factor using a pre-shared relay instruction.

8. A key management system for AI-to-AI communication sessions, the system comprising:

a control plane service configured to receive registration requests from AI nodes and to generate, for each session, an ephemeral session key bundle comprising an encryption matrix K , a decryption matrix K_{inv} , a watermark seed, a session identifier, and a session expiry time;

distribute K and the watermark seed to transmitting nodes;

distribute K_{inv} and the watermark seed to receiving nodes;

invalidate sessions upon expiry such that subsequent payloads using expired keys fail authentication; and

upon request, generate pairs of orthogonal projection matrices (W_A , W_B) for multi-signal isolation sessions.

9. A machine-readable non-transitory medium storing instructions that, when executed by a processor, implement a communication protocol comprising a Protocol Buffer message schema defining a payload message containing: a binary-packed floating-point vector field; an embedding dimension field; a session identifier field; a transmission mode field selecting among at least direct, isolation, and superposition modes; a monotonically increasing sequence number field; and a watermark vector field; wherein said message is

transmitted as a binary payload over a gRPC streaming channel without text encoding of the vector data.

ABSTRACT

The CHORUS Fabric is a machine-to-machine communication protocol for artificial intelligence systems that transmits high-dimensional float32 tensor vectors directly over bidirectional gRPC streams, bypassing text tokenization. The system comprises: (1) a tensor multiplication cipher wherein signals are encrypted and decrypted by matrix multiplication with session-specific key matrices generated via QR decomposition; (2) a rolling neural watermark that injects a SHA-256-derived deterministic unit vector into each payload before encryption, providing per-message authentication bound to session identity and sequence number; (3) an orthogonal isolation mode enabling two AI agents to share a channel without signal crosstalk using orthogonal projection matrix pairs; (4) a holographic superposition mode arithmetically combining multiple agent signals into a single collective transmission vector; (5) a relay node that amplifies ciphertext by scalar multiplication without holding a decryption key, preserving end-to-end confidentiality while logging SHA-256 payload fingerprints for audit; and (6) a control plane key issuance service distributing ephemeral session key bundles with configurable time-to-live. Transatlantic deployment on Azure Container Instances (US East to Germany West Central) demonstrated 179 millisecond p50 round-trip latency with 100% watermark verification across 7,766 transmissions, and $4.45\times$ lower bandwidth consumption than equivalent HTTP/REST JSON communication.

INVENTOR INFORMATION

(Complete this section and transfer to USPTO Form SB/16 when filing at patentcenter.uspto.gov)

Inventor Full Legal Name	Amin Parva
---------------------------------	------------

City of Residence	Mission Viejo
--------------------------	---------------

State / Country of Residence	CA, United States
-------------------------------------	-------------------

Citizenship	United States
--------------------	---------------

Mailing Address	39 Aliso Ridge Loop, Mission Viejo, CA 92691, United States
------------------------	---

Email	parvaamin@gmail.com
--------------	---------------------

Entity Status	☒ Micro Entity (\$160)
----------------------	--

Title of Invention	The Chorus Fabric: High-Dimensional Signal Orchestration for Machine-to-Machine Communication
---------------------------	--